

Rails Ticket Booking Application

Objective

Design and architect a Ruby on Rails application to simulate a simplified bus ticket booking system, focusing on **robustness, concurrency handling, and adherence to best practices**.

Your solution must incorporate the following core business logic and address the highlighted technical constraints:

1. Core Functionality & User Flow

- **Authentication:**
 - Implement a basic email-only signup/login flow.
- **Trip Search & Filter:**
 - Allow passengers to search for trips using
 - From City
 - To City
 - Travel Date
 - Implement **advanced filtering** by
 - operator rating
 - price range
 - bus type (AC/Non-AC, Sleeper/Seater)
 - amenities (Wi-Fi, etc.).

- **Seat Selection & Temporary Hold (Concurrency Critical):**
 - Users must be able to select one or more available seats for a specific trip.
 - Upon submission, the system must create a **Temporary Hold** record with a **5-minute expiry**.
 - **Concurrency Rule:**
 - **Crucially**, no two users must be allowed to hold the same seat simultaneously. You must ensure **no overselling occurs**.
- **Booking Confirmation (Idempotency Critical):**
 - Once a user has a valid hold, clicking "Confirm Booking" converts the Hold into a booking.
 - The system must satisfy the **Idempotency Requirement**: Repeated attempts (e.g., refreshing the page) must **NOT** create duplicate bookings.
- **Booking Modification:**
 - Users must be able to **reschedule** a confirmed booking to a different date or time (for the same route/operator).
- **Booking Cancellation:**
 - Users can cancel a confirmed booking only if the cancellation is requested **at least 1 hour before the trip departure**.

- Implement a simplified refund rule: refund = ticket price - ₹50 (fixed flat cancellation fee).

2. Technical & Architecture Requirements

Your solution must demonstrate expertise in these specific Rails components:

- **Database:** Utilize **PostgreSQL** for all persistence.
- **Concurrency Handling:** Detail the specific database and application-level strategy you will use to enforce the **single-user-per-seat-hold** rule (e.g., pessimistic locking, unique constraints, or transactional integrity).
- **Background Jobs:** Utilize **Sidekiq or ActiveJob** to implement the required **Hold Expiry Logic**, automatically releasing seats when the 5-minute hold window passes.
- **Architecture:** Leverage standard Rails conventions, including **Service Objects** or **Form Objects** to encapsulate the complex business logic (e.g., in `BookingConfirmationService` and `CancellationService`).
- **Performance:** Implement a basic **caching strategy** (e.g., using `Rails.cache`) for search results to improve performance, and explain the mechanism to **expire the cache** when a booking or cancellation changes seat availability.
- **Testing:** Use **RSpec** to cover critical business logic, specifically testing: **Seat hold creation, Expiry logic, Double booking prevention, and Cancellation logic.**

Based on these requirements, outline the complete Model schema (Trip, Seat, Hold, Booking, etc.) and the high-level code structure (Controller actions, Service

Objects, and the critical concurrency mechanism) you would implement to ensure the application is production-ready, highly concurrent, and idempotent